COMPARAÇÃO EXPERIMENTAL DE BFS E DFS NO SLIDING PUZZLE

PERGUNTA DE PESQUISA

Como BFS e DFS, com controle de estados repetidos, diferem em tempo de execução, número de nós gerados e tamanho máximo da fronteira ao resolver Sliding Puzzles 3×3 e 4×4, sob os mesmos limites de recursos?

EXPERIMENTOS

Foi utilizada a biblioteca da Equipe A, obtida de GustavoFelipeM/search_lib. As abstrações State, Action, Problem, SearchNode e as métricas foram mantidas. Os algoritmos foram adaptados de tree-search para graph-search: os estados são registrados ao entrar na fronteira e não são inseridos novamente. BFS utiliza uma fila FIFO; DFS, uma pilha LIFO. Ambos aplicam o mesmo teste de objetivo ao remover um estado da fronteira e visitam os sucessores na ordem cima, baixo, esquerda e direita. Cada movimento tem custo 1. DLS não faz parte do estudo.

O fator principal é a estratégia (BFS ou DFS), cruzada com a dimensão do problema (3×3 ou 4×4). O comprimento do Random Walk funciona como estrato de geração das entradas: 5, 10, 15, 20, 30 e 40 movimentos. Para cada combinação de tamanho e comprimento, foram usadas 20 sementes, de 42 a 61. Ao todo, são 240 instâncias pareadas e 480 buscas. Cada instância foi aplicada aos dois algoritmos sem alterações; a ordem de execução alternou entre as rodadas. As 20 rodadas correspondem a sementes diferentes, não a medições repetidas do mesmo tabuleiro.

Os estados iniciais foram gerados a partir do objetivo ordenado, com o espaço vazio no canto inferior direito, escolhendo-se uniformemente um movimento legal a cada passo. Todos os estados são solucionáveis. Retornos e ciclos são permitidos; portanto, o comprimento do passeio não corresponde à distância ótima. Foram mantidas 8 instâncias que já estavam no objetivo. Como as sementes foram reutilizadas, comprimentos diferentes podem compartilhar prefixos do passeio; assim, as observações não devem ser tratadas como independentes entre todos os estratos.

O orçamento foi de 30.000 estados admitidos e 2 segundos por busca. A interrupção por número de nós ocorre quando há uma tentativa de admitir o próximo estado. Assim, uma solução que ainda esteja na fronteira pode não ser encontrada antes da interrupção. As execuções interrompidas foram mantidas nos resultados, com custo vazio e status explícito. A coleta foi realizada no Windows 11, com Python 3.14.6, em 19/10/2026. Trata-se de uma execução local, sem isolamento da carga do sistema; não foram feitas repetições temporais nem testes de significância.

Nós gerados são os estados distintos admitidos, incluindo a raiz; o tamanho máximo da fronteira é o pico de nós que aguardam exploração. O tempo de relógio foi medido com perf_counter, excluindo a geração e a exportação, mas incluindo a instrumentação por tracemalloc. O pico de

memória Python, os nós expandidos e o custo também constam no CSV. A memória Python não equivale ao RSS total. As figuras usam as médias de todas as execuções, inclusive as interrompidas; as faixas mostram os percentis 25–75, e não intervalos de confiança. O eixo vertical das três métricas principais é logarítmico.

RESULTADOS

Tabuleiro	Busca	Soluções	Tempo médio (s)	Nós gerados médios	Fronteira máxima média
3×3	BFS	116/120	0.0342	1722.1	607.1
3×3	DFS	16/120	0.4887	26021.0	11221.6
4×4	BFS	97/120	0.1516	8122.3	4080.9
4×4	DFS	4/120	0.5637	29065.5	14647.6

No total, BFS resolveu 213/240 instâncias, e DFS resolveu 20/240. Os status registrados foram: node_limit: 247; solved: 233. Atingir um limite indica que não houve solução dentro do orçamento, não que a instância seja insolúvel. O teto de nós também comprime as diferenças entre execuções difíceis: valores próximos de 30.000 refletem o protocolo de interrupção.

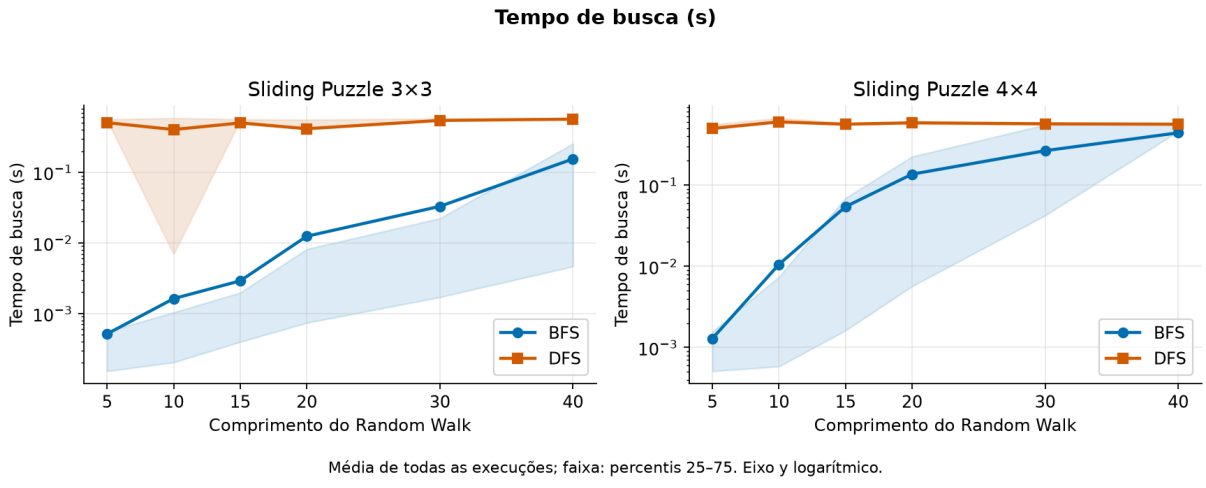


Figura 1 — Tempo. As médias devem ser interpretadas junto à taxa de sucesso: elas medem o esforço até a solução ou a interrupção, não o tempo necessário para resolver todas as entradas. A instrumentação e a carga do computador afetam os tempos absolutos.

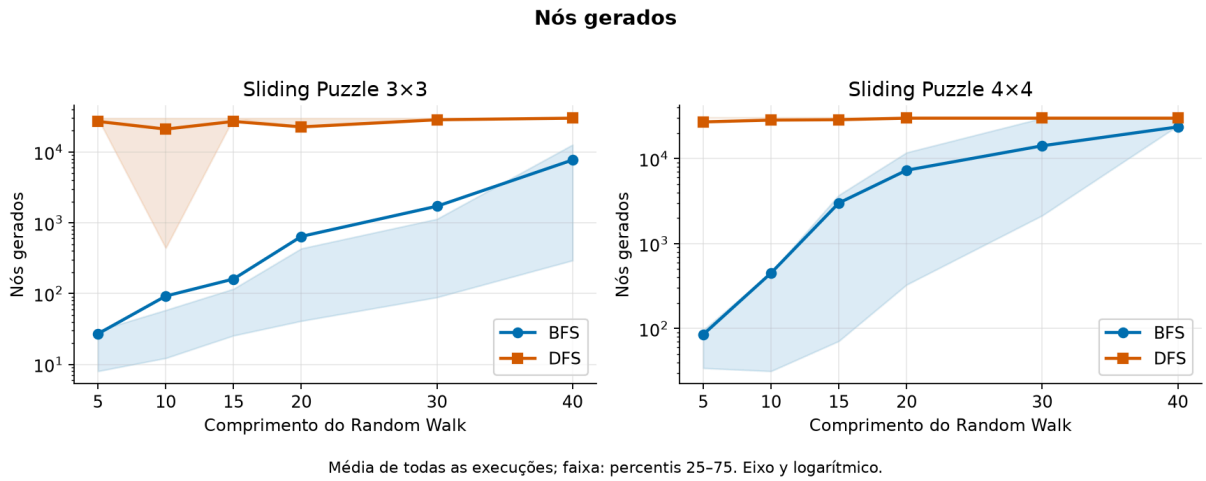


Figura 2 — Nós gerados. O gráfico evidencia o custo de exploração e a saturação no limite. O DFS pode percorrer ramos longos mesmo quando o estado inicial foi obtido com poucos movimentos; a BFS explora por camadas e se beneficia de soluções rasas.

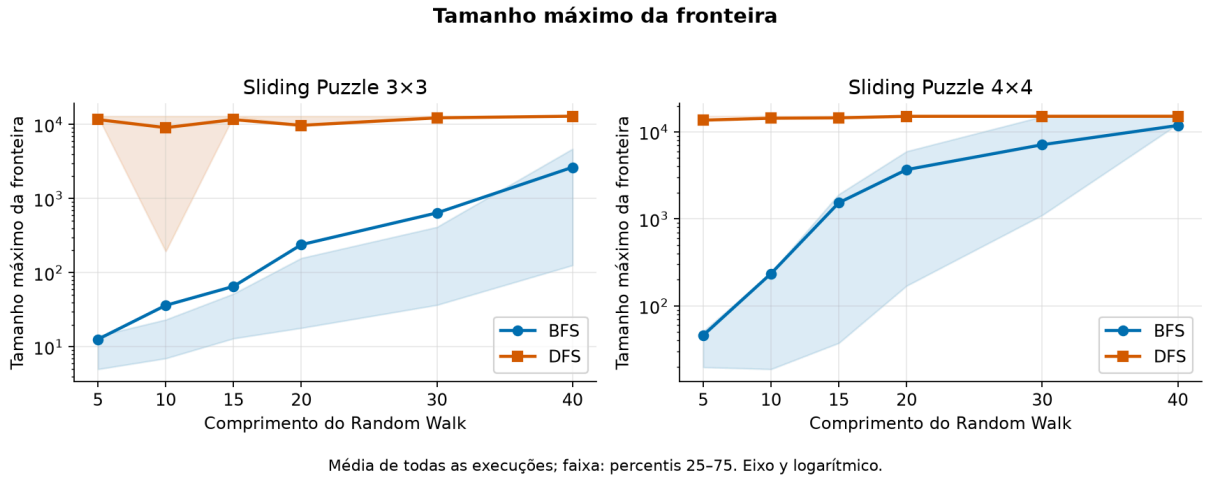


Figura 3 — Fronteira. O conjunto de estados descobertos e os vínculos de ancestralidade também consomem memória. Portanto, uma fronteira menor não implica necessariamente menor uso total de memória; não se deve aplicar diretamente a expectativa de memória do DFS em árvore a esta implementação em grafo.

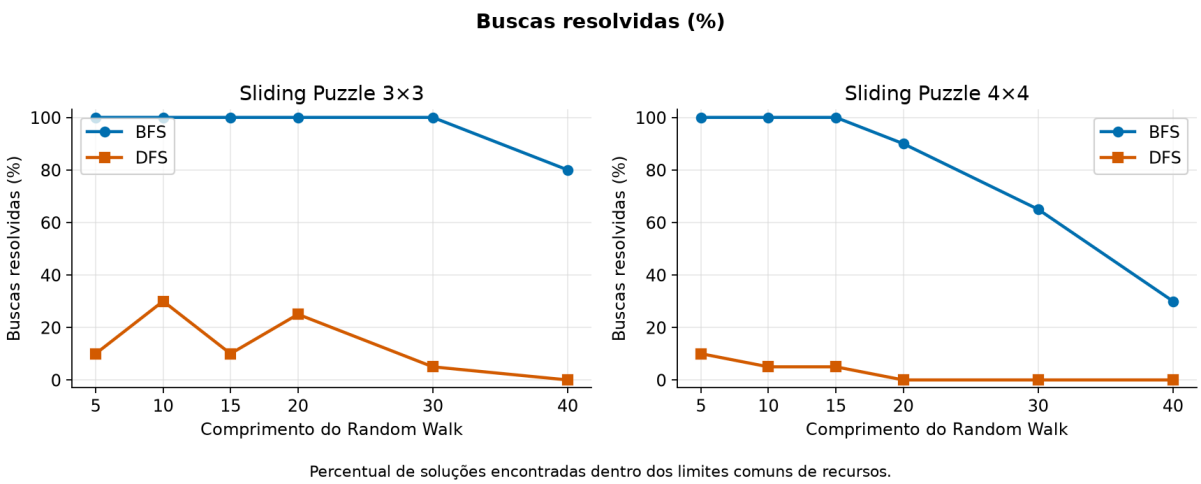


Figura 4 — Sucesso. Complementa a interpretação das três métricas. Não se observa necessariamente um crescimento monotônico da dificuldade conforme aumenta o comprimento do passeio, pois são permitidos retornos.

Houveram 20 pares resolvidos por ambos os algoritmos; em 12 deles, o custo foi igual. Os custos só devem ser comparados nesses pares, evitando-se comparar médias calculadas sobre conjuntos de sucesso diferentes. Com custos unitários, a BFS encontra caminhos mínimos quando conclui a busca; a DFS não oferece essa garantia.

REPRODUÇÃO E ARQUIVOS

```
python main.py benchmark --sizes 3 4 --depths 5 10 15 20 30 40 --trials 20 --seed 42 --max-nodes 30000 --time-limit 2 --output results/d1.csv
python -m pip install -r requirements-plots.txt
python plot_results.py results/d1.csv --output results/figures
python -B -m unittest discover -s tests -v
```

Dados: results/d1.csv. Resumo por configuração: results/d1.md. Ambiente e hashes: results/d1_metadata.json. Origem e adaptações: SEARCH_LIB_SOURCE.txt. Os oito testes automatizados passaram; a auditoria dos dados confirmou 240 pares com entradas e limites idênticos e 480 registros completos.

CONCLUSÃO E LIMITES

Neste conjunto de entradas e com o orçamento adotado, a BFS apresentou maior cobertura de soluções do que a DFS. O resultado é compatível com o benefício da exploração por camadas em

estados gerados por passeios relativamente curtos. Mesmo evitando ciclos, a DFS pode consumir o orçamento em ramos profundos antes de alcançar o objetivo.

Essa conclusão está restrita às versões graph-search, à ordem dos movimentos, às sementes, aos dois tamanhos e aos limites descritos. O experimento não estabelece superioridade universal da BFS, não avalia tree-search e não mede o custo completo das execuções interrompidas. O Random Walk também não produz uma amostragem uniforme do espaço de estados. Uma extensão possível seria variar a ordem dos sucessores ou o orçamento, mantendo as mesmas instâncias pareadas.